

Empirical Boundaries of Lightweight Process Supervision: A Critical Replication and Case Study of Candidate Reranking on GSM8K

Xiaoci Zhang Yunhe Dong

Department of Computational Linguistics

Heidelberg University

{xiaoci.zhang, yunhe.dong}@stud.uni-heidelberg.de

Abstract

Process-Supervised Verifiers (PSVs) have emerged as a prominent paradigm to mitigate hallucinations in complex reasoning chains by awarding credit or assigning penalties to intermediate steps. Recently, the AutoPSV framework demonstrated that step-level supervision can be automated through outcome-supervised confidence variations (Δ_{conf}^t), sidestepping costly Monte Carlo Tree Search (MCTS) or human step annotations. In this work, we conduct a rigorous, resource-constrained replication study evaluating whether this automated process supervision paradigm translates effectively to sub-billion parameter backbones. Specifically, we implement an adaptation pipeline utilizing Qwen2.5-0.5B-Instruct optimized via 4-bit NormalFloat quantization and Low-Rank Adaptation (LoRA), trained over 400 step-annotated mathematical reasoning trajectories (5,302 prefix states) on an HPC debugging partition. Evaluating across a strictly separated test set of 200 out-of-sample GSM8K problems (1,000 candidate solutions), our empirical findings yield a concrete negative result: both Min aggregation (34.0%, 95% CI: [27.8%, 40.8%]) and Product aggregation (33.5%, 95% CI: [27.3%, 40.3%]) underperform unsupervised Majority Voting (42.0%, 95% CI: [35.4%, 48.9%]) by 8.0 and 8.5 percentage points, respectively, against a theoretical Oracle ceiling of 59.5%. We present an exhaustive root-cause error taxonomy, including length-induced tail score degradation, step-segmentation sensitivity, parsing failure penalties (15.2% unparsed solutions), and model capacity constraints. This study delivers quantitative boundary conditions and reproducible benchmarks for lightweight verification systems.

1 Introduction

Large Language Models (LLMs) have demonstrated substantial capabilities in multi-step deductive and arithmetic reasoning (Wei et al., 2022;

Brown et al., 2020). Nonetheless, due to their autoregressive decoding nature, intermediate errors inevitably cascade, culminating in spurious derivations and invalid conclusions. To enhance test-time execution, sampling multiple candidate trajectories per question has become standard practice. The operational challenge consequently shifts from generation to selection: How can an evaluation agent reliably select the ground-truth trajectory among multiple plausible candidates?

A primary baseline is unsupervised Self-Consistency or Majority Voting (Wang et al., 2023b), which aggregates candidate solutions via modal final answers. Despite its simplicity and competitive empirical performance, Majority Voting suffers from systematic structural vulnerabilities: when multiple sampled trajectories share an identical invalid assumption, arithmetic error, or systematic prompt bias, modal aggregation collapses onto an incorrect consensus.

To overcome the blind spots of majority voting, verification approaches have bifurcated into Outcome-Supervised Verifiers (OSVs) (Cobbe et al., 2021) and Process-Supervised Verifiers (PSVs) (Lightman et al., 2023; Uesato et al., 2022). While OSVs only assign scalar rewards based on the terminal correctness of an entire chain, PSVs assess individual reasoning steps, pinpointing the exact emergence of logical or calculation errors. Historically, training PSVs necessitated either prohibitive human annotation campaigns (Lightman et al., 2023) or massive Monte Carlo Tree Search (MCTS) rollouts (Wang et al., 2023a), which can require upwards of 10 to 12 GPU-days for moderately sized benchmarks.

To circumvent this annotation bottleneck, Lu et al. (2024) introduced AutoPSV, demonstrating that an outcome-trained verifier’s internal confidence variations across adjacent steps (Δ_{conf}^t) can automatically generate step-level supervision targets without human intervention or future-state roll-

outs.

While original investigations utilized multi-billion parameter foundations (e.g., Mistral-7B, Mixtral-8x7B, Qwen-72B) trained over hundreds of thousands of full-parameter tokens on multi-GPU server clusters, real-world deployment frequently encounters stringent computational limitations. Computational linguistics research and edge deployments often demand running verification on commodity hardware, high-performance computing (HPC) short-queue debugging partitions, or small-footprint models. This raises fundamental scientific and engineering questions:

- RQ1. Feasibility and Faithfulness:** Can the AutoPSV confidence-drop annotation paradigm be successfully mapped onto an ultra-compact language model (e.g., Qwen2.5-0.5B) under Parameter-Efficient Fine-Tuning (LoRA) and 4-bit quantization?
- RQ2. Verification Superiority:** Under low-resource constraints, can a lightweight process verifier reliably outperform the standard majority voting consensus in mathematical candidate reranking?
- RQ3. Pathology and Failure Modes:** What specific computational bottlenecks, score drift phenomena, and token parsing failures govern the performance delta between step verification and frequency consensus?

To address these questions, we construct a fully controlled, auditable, and reproducible candidate reranking benchmark on the GSM8K benchmark (Cobbe et al., 2021). We strictly separate training, diagnostic validation, and formal testing sets to prevent data leakage. Using a 4-bit quantized Qwen2.5-0.5B-Instruct backbone fine-tuned on bwUniCluster HPC debugging nodes, we evaluate reranking performance across 200 held-out problems (1,000 generated candidate solutions).

Our experiments reveal an informative and robust negative result: neither Min nor Product process aggregation surpasses the Majority Voting baseline. Through exhaustive manual trajectory audit, regression checks, and step-level score decomposition, we identify the exact mechanistic failures responsible for this gap. We show that lightweight value heads suffer severely from length-induced tail score degradation and stylistic vulnerability, which paradoxically causes them to assign higher scores

to concise incorrect solutions than to exhaustive correct ones.

2 Related Work

2.1 Reasoning and Test-Time Search in LLMs

Chain-of-Thought (CoT) prompting (Wei et al., 2022) significantly improves complex multi-step reasoning by explicitly instructing models to produce intermediate verbal derivations prior to the terminal answer. Subsequent efforts expanded linear CoT into deliberate tree-structured or graph-structured search spaces, such as Tree-of-Thoughts (Yao et al., 2023) and program-aided generation (Gao et al., 2023). However, greedy decoding often yields suboptimal derivations, motivating multi-path sampling strategies (Pass@K) coupled with selection mechanisms.

2.2 Outcome vs. Process Supervision

The selection of candidate solutions has traditionally relied on outcome-based verification (OSV) (Cobbe et al., 2021; Yu et al., 2023), wherein a reward head predicts whether a complete solution trajectory S_i leads to the correct final answer $y_i \in \{0, 1\}$. Yu et al. (2023) proved that an OSV trained via mean squared error (MSE) over step prefixes mathematically approximates the conditional probability of reaching a correct solution from that intermediate state, $f_\theta(S^{(1:t)}; q) \approx p(\hat{a} | S^{(1:t)}, q)$.

Despite theoretical consistency, OSVs suffer from outcome bias: they reward incorrect reasoning paths that arrive at the correct terminal answer by chance (false positives), and harshly penalize reasoning paths that execute multiple flawless reasoning steps but fail due to a trivial terminal calculation slip (false negatives). To provide fine-grained localization, Process-Supervised Verifiers (PSVs) were introduced (Uesato et al., 2022; Lightman et al., 2023). Lightman et al. (2023) showed that PSVs substantially outperform OSVs across math competition problems (PRM800K). However, PRM800K demanded hundreds of hours of expert human annotation, creating severe scalability bottlenecks.

2.3 Automated Process Annotation and AutoPSV

To bypass manual labeling, model-induced process supervision emerged. Wang et al. (2023a) developed Math-Shepherd, using Monte Carlo rollouts to estimate step completion probabilities. Similarly,

Wang et al. (2024) investigated automated MCTS rollouts. However, MCTS methods impose massive compute costs: rolling out 8 paths per step across 15,000 problems requires billions of generated tokens and upwards of two weeks of execution on enterprise clusters (Lu et al., 2024).

Addressing this efficiency gap, Lu et al. (2024) introduced AutoPSV. AutoPSV trains an initial outcome-supervised value head and detects relative drops in model confidence across adjacent steps (Δ_{conf}^t). If the relative confidence drop surpasses a designated threshold θ , the step is labeled incorrect, and all subsequent steps are marked invalid under the first-error assumption. AutoPSV reduced token annotation costs by over 95% compared to MCTS while matching or exceeding verifier performance on Mistral-7B and Qwen-72B backbones. Our work directly examines whether this premise remains viable when scaled down to ultra-lightweight architectures.

3 Methodology

3.1 Problem Formulation

Let $q \in \mathcal{Q}$ denote a natural language mathematical problem drawn from the question distribution. An autoregressive language model acts as a candidate generator G_ϕ , producing K candidate solutions $S_q = \{S_1, S_2, \dots, S_K\}$. Each candidate solution S_i comprises a sequence of m_i discrete reasoning steps, denoted as:

$$S_i = (s_i^1, s_i^2, \dots, s_i^{m_i}) \quad (1)$$

Let $S_i^{(1:t)} = (s_i^1, \dots, s_i^t)$ represent the prefix trajectory up to step t . Each candidate terminates with an extracted numerical answer $\hat{a}_i = \text{Extract}(S_i)$. The ground-truth answer is denoted by a_q^* . The candidate’s outcome correctness is binary: $y_i = \mathbb{I}(\hat{a}_i = a_q^*)$.

The objective of candidate reranking is to define a scoring function $\mathcal{R}(S_i; q)$ parameterized by a verifier such that selecting the highest-scoring candidate maximizes the probability of correctness:

$$i^* = \arg \max_{i \in \{1, \dots, K\}} \mathcal{R}(S_i; q) \quad (2)$$

3.2 Automated Process Annotation via Δ_{conf}^t

Following Lu et al. (2024), step-level supervision targets are automatically induced from an outcome-supervised scoring function f_θ . The relative confidence change between consecutive prefix states is

defined as:

$$\Delta_{\text{conf}}^t = \frac{f_\theta(S^{(1:t+1)}; q) - f_\theta(S^{(1:t)}; q)}{f_\theta(S^{(1:t)}; q)} \quad (3)$$

Intuitively, if step $t + 1$ introduces a mathematical or conceptual error, the probability of reaching a correct conclusion collapses, resulting in a sharply negative Δ_{conf}^t .

Under the first-error location hypothesis (Uesato et al., 2022), reasoning trajectories are assumed to be irrecoverable once a fatal error occurs. Given a variation threshold $\theta \in [-1.0, 0.0]$, the step-level pseudo-labels $y^t \in \{0, 1\}$ are assigned as:

$$y^t = \begin{cases} 1, & \text{if } \forall j \leq t, \Delta_{\text{conf}}^j > \theta \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

Prior to formal test-set evaluation, we analyzed label distributions under three candidate thresholds: $\theta \in \{-0.50, -0.35, -0.20\}$. While stricter thresholds ($\theta = -0.20$) capture subtle deviations, they heavily over-penalize stylistic transitions. In contrast, $\theta = -0.50$ balances error detection with stability. We accordingly freeze $\theta = -0.50$ across all training trajectory generations.

3.3 Parameter-Efficient Architecture

To assess verifier viability under extreme resource constraints, we adopt Qwen2.5-0.5B-Instruct as the base backbone. To enable execution within restricted GPU memory envelopes without out-of-memory (OOM) faults, we introduce two architectural adaptations:

- **4-bit NormalFloat Quantization (NF4):** Base model parameters $W \in \mathbb{R}^{d_1 \times d_2}$ are frozen and quantized into 4-bit representations with double quantization to minimize quantization-induced error.
- **Low-Rank Adaptation (LoRA):** Trainable rank-decomposition matrices $A \in \mathbb{R}^{r \times d_2}$ and $B \in \mathbb{R}^{d_1 \times r}$ are injected into attention projection layers (W_q, W_v):

$$W' = W + \frac{\alpha}{r} B A \quad (5)$$

where rank $r = 8$ and scaling factor $\alpha = 16$.

- **Scalar Value Head:** A linear projection head $H \in \mathbb{R}^{d \times 1}$ is attached to the final hidden state of the end-of-step delimiter tokens, mapped through a sigmoid activation to yield predicted step probabilities $v_t \in [0, 1]$.

The verifier is trained using Mean Squared Error (MSE) loss over all prefix steps across training trajectories $\mathcal{T}$:

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{T}|} \sum_{S_i \in \mathcal{T}} \frac{1}{m_i} \sum_{t=1}^{m_i} \left(f_{\theta}(S_i^{(1:t)}; q) - y_i^t \right)^2 \quad (6)$$

3.4 Candidate Score Aggregation

Given predicted step probabilities $\{v_1, v_2, \dots, v_{m_i}\}$ for candidate S_i , we implement two distinct mathematical aggregation paradigms:

1. **Min Aggregation ($\mathcal{R}_{\min}$):** Guided by the weakest-link principle, the solution’s quality is bounded by its lowest-scoring intermediate transition:

$$\mathcal{R}_{\min}(S_i) = \min_{1 \leq t \leq m_i} v_t \quad (7)$$

2. **Product Aggregation ($\mathcal{R}_{\text{prod}}$):** Guided by the joint independent probability formulation, the solution’s quality is the product of all step-level transition likelihoods. To ensure numerical stability, this is evaluated via log-sum-exp transformation:

$$\mathcal{R}_{\text{prod}}(S_i) = \sum_{t=1}^{m_i} \log(v_t + \epsilon) \quad (8)$$

where $\epsilon = 10^{-7}$ prevents domain singularity.

4 Experimental Setup

4.1 HPC Infrastructure and Job Scheduling

All training, prefix extraction, and evaluation workloads were orchestratedd on the bwUniCluster 3.0 supercomputing infrastructure. To adhere to strict operational guidelines:

- **Execution Environment:** Workloads were allocated on accelerated debugging partitions (dev_gpu_4_a100) equipped with NVIDIA A100 Tensor Core GPUs (40GB VRAM) and AMD EPYC 7742 processors.
- **Slurm Job Encapsulation:** Non-interactive tasks were submitted via automated Slurm batch scripts with strict walltime limits ($T \leq 01:00:00$) and 30GB host memory allocations.
- **Cache Isolation:** All Hugging Face and PyTorch runtime caches were redirected to non-volatile local scratch partitions (`/scratch/$USER`) to prevent home-directory disk quota saturation.

Hyperparameter	Assigned Value
Backbone Model	Qwen2.5-0.5B-Instruct
Quantization Precision	4-bit NF4 (Double Quant)
LoRA Rank (r)	8
LoRA Alpha (α)	16
LoRA Target Modules	W_q, W_v
LoRA Dropout	0.05
Trainable Parameters	1.05M / 492M (0.21%)
Optimizer	AdamW
Learning Rate	2×10^{-4}
LR Schedule	Linear with 10% Warmup
Batch Size (Per Device)	4
Gradient Accumulation Steps	4
Total Update Steps	30
Training Trajectories	400
Prefix Sample Count	5,302
Labeling Threshold (θ)	-0.50

Table 1: Training configurations for lightweight process verifier on the HPC accelerated debugging node.

4.2 Training Protocol and Hyperparameters

The training dataset consists of 400 distinct mathematical reasoning trajectories generated from GSM8K training prompts, decomposed into 5,302 prefix-target pairs. Verifier optimization parameters are specified in Table 1. The value head was trained with an effective batch size of 16 across 30 gradient update steps, using the AdamW optimizer with decoupled weight decay ($\lambda = 0.01$) and linear learning rate warmup.

4.3 Strict Data Separation and Generation Parameters

To guarantee rigorous empirical evaluation:

- **Zero Contamination:** A formal test split of 200 questions was sampled from the GSM8K test set, verified via SHA-256 hash auditing (675d924f...) to ensure absolute disjointness from all local training trajectories, diagnostic sets, and development prompts.
- **Sampling Configuration:** For each problem, 5 candidate solutions were generated using the frozen Qwen2.5-0.5B-Instruct model under stochastic decoding: temperature $T = 0.7$, top- $p = 0.9$, with maximum generated tokens constrained to 768.
- **Prompt Formatting:** Solutions were enforced with the suffix: *"Solve with at most six short reasoning steps. End with a separate line exactly: #### ;number;. Do not write anything after that line."*

- **Throughput:** Generation of all 1,000 candidate chains required 3,775.73 seconds ($\approx$ 62.9 minutes) of sustained compute.

4.4 Checkpoint Persistence and Reload Audit

In machine learning deployments, non-deterministic model serialization or adapter re-initialization frequently introduces silent inference discrepancies. We executed an explicit post-training persistence audit:

$$\text{Error}_{\text{reload}} = \max_{x \in \mathcal{D}_{\text{audit}}} |f_{\theta}^{\text{saved}}(x) - f_{\theta}^{\text{reloaded}}(x)| \quad (9)$$

On identical input representations, the reloaded LoRA value head produced an initial score of 1.2109375 before saving and exactly 1.2109375 after re-instantiation, establishing an absolute reload error of 0.0.

4.5 Answer Extraction and Penalization Protocol

Numerical answers were parsed from terminal candidate text using standardized regular expression matching against the #### token delimiter. Among the 1,000 generated candidate solutions, exactly 152 candidates failed answer extraction due to format deviations, early generation truncation, or malformed numerical expressions.

Standardized Evaluation Rule: All 152 unextractable candidates were treated as invalid ($y = 0$). In Majority Voting, unextractable candidates were discarded from the voting pool, allowing valid candidates to establish consensus. In verifier reranking, unextractable candidates were permitted to compete based on their step scores; if selected, they were evaluated as incorrect.

5 Quantitative Results

5.1 Main Benchmark Performance

Table 2 reports the performance of the lightweight process-supervised verifier across the 200 formal test problems against baseline selection paradigms, accompanied by 95% Wilson score confidence intervals.

The empirical results document that neither process verification approach exceeds the consensus baseline:

- **Majority Voting** achieves 42.0% accuracy (84/200), demonstrating remarkable robustness

Method	Correct	Acc. (%)	95% CI	Δ vs. Maj.
Min ($\mathcal{R}_{\text{min}}$)	68 / 200	34.0%	[27.8%, 40.8%]	-8.0%
Product ($\mathcal{R}_{\text{prod}}$)	67 / 200	33.5%	[27.3%, 40.3%]	-8.5%
Majority Vote	84 / 200	42.0%	[35.4%, 48.9%]	—
Oracle (Pass@5)	119 / 200	59.5%	[52.6%, 66.1%]	+17.5%

Table 2: Quantitative performance across 200 GSM8K test instances (1,000 candidates). Wilson score intervals computed at $\alpha = 0.05$. Unextractable candidates (152/1000) are scored as incorrect.

despite relying solely on terminal answer frequencies without any step evaluation.

- **Min Aggregation** achieves 34.0% accuracy (68/200), trailing Majority Voting by 8.0 percentage points.
- **Product Aggregation** achieves 33.5% accuracy (67/200), trailing Majority Voting by 8.5 percentage points.
- **Oracle Ceiling:** The candidate generation pool contains at least one correct solution for 119 out of 200 problems (59.5%). This establishes that the candidate pool was sufficiently diverse to support up to a 17.5% improvement over majority voting, confirming that verifier underperformance stems from selection errors rather than generator bankruptcy.

5.2 Statistical Hypothesis Testing

To ascertain whether the 8.0% performance deficit between Min aggregation and Majority Voting is statistically meaningful, we conduct a two-sided McNemar’s test on paired nominal response vectors. McNemar’s test statistic with continuity correction yields $\chi^2 = 5.92$ ($p = 0.0149$). At the standard significance threshold $\alpha = 0.05$, the null hypothesis of equivalent performance is rejected. This confirms that the observed underperformance of the lightweight verifier is statistically significant and not an artifact of random test sampling.

5.3 Cross-Method Discrepancy Matrix

To uncover the inter-dependencies between frequency consensus and process verification, we decompose the 200 test problems into a mutually exclusive categorical cross-tabulation in Table 3.

The cross-tabulation highlights two critical behavioral dynamics:

1. **Positive Recovery (11 Cases, 5.5%):** Instances where the majority of generated candidates con-

Categorical Regime	Count	Ratio (%)
Majority Correct $\cap$ Min Correct	57	28.5%
Majority Wrong $\cap$ Min Wrong	105	52.5%
Positive Recovery (Maj. 0, Min 1)	11	5.5%
False Attribution (Maj. 1, Min 0)	27	13.5%
Oracle Correct $\cap$ Maj. 0 $\cap$ Min 0	23	11.5%
Oracle Wrong (Zero Valid Candidates)	81	40.5%

Table 3: Decomposition of decision dynamics between Majority Voting and Min Aggregation across 200 test questions.

	Product = 0	Product = 1	Total
Min = 0	125	7	132
Min = 1	8	60	68
Total	133	67	200

Table 4: Confusion matrix comparing candidate correctness between Min and Product aggregation functions.

verged on an invalid consensus, but the Min verifier correctly identified and elevated a minority valid candidate.

- False Attribution Degradation (27 Cases, 13.5%):** Instances where the majority consensus was correct, but the verifier overruled it in favor of an invalid candidate with deceptively high step scores.

Because False Attribution Degradation occurrences outnumber Positive Recovery occurrences by a factor of $2.45 \times$ (27 vs. 11), the net operational effect of verifier reranking is negative, resulting in a net loss of 16 solved problems.

5.4 Min vs. Product Decision Dynamics

Table 4 investigates the decision correlation between Min and Product aggregation functions. Min and Product selected the exact same candidate in 124 out of 200 problems (62.0%). Across the entire test split, 60 problems were solved correctly by both methods, while 125 problems were missed by both. Min uniquely solved 8 problems where Product failed, and Product uniquely solved 7 problems where Min failed.

5.5 Reasoning Chain Length Bias

To assess whether aggregation functions exhibit systemic length bias, we compute the distribution of reasoning step lengths across selected candidates:

- Candidates selected by Min Aggregation possess an average step length of 11.52 ± 4.45 steps (range: [4, 26]). Candidates that were correct averaged 10.60 ± 3.92 steps, whereas candidates that were incorrect averaged 11.99 ± 4.63 steps.
- Candidates selected by Product Aggregation possess an average step length of 10.28 ± 3.77 steps (range: [3, 25]).
- Paired step-length difference ($\text{Steps}_{\text{min}} - \text{Steps}_{\text{prod}}$) shows a mean delta of +1.24 steps (median: 0.0, max: +15.0).

Because Product aggregation compounds probabilities via logarithmic summation $\sum \log(v_t)$, every additional step inevitably introduces an additive penalty ($\log v_t \leq 0$). Consequently, Product aggregation systematically penalizes longer reasoning chains, frequently truncating candidate selection toward brief, underspecified derivations.

6 Qualitative Error Taxonomy and Case Studies

To understand the precise mechanistic drivers behind verifier failures, we conducted a manual audit across the 38 divergent test problems (11 positive recoveries, 27 degradations). We formulate a four-part qualitative error taxonomy.

6.1 Mechanistic Audit of Contrasting Cases

Table 5 provides an in-depth trace audit of two primary representative cases drawn from our formal test logs.

Case 1: Recovery Over Invalid Majority Consensus (gsm8k-test-118) The problem states: *"A customs officer counted 2 containers of imported vehicles, each having 5 vehicles inside. The next day, more containers were brought in, and the total number of vehicles became 30. Calculate the number of containers imported on the second day."*

- Majority Trajectory (c0):** The generator established an initial vehicle count of 10. However, in Step 4, it defined: *"Let x be the number of additional containers imported... Therefore, the equation becomes $10 + x = 30 \Rightarrow x = 20$."* It conflated vehicles with containers, arriving at $x = 20$. Because three independent candidates shared this identical arithmetic leap, Majority Voting collapsed onto 20.0.

Candidate ID	Critical Step / Error Mechanism	Steps	$\mathcal{R}_{\min}$	Parsed / Gold
Case 1: Successful Minority Recovery (<code>gsm8k-test-118</code> , Gold Answer: 4)				
118-c0 (Majority, 3 votes)	<i>Conceptual confusion</i> : Set $10 + x = 30 \Rightarrow x = 20$ (confused container count with vehicle delta). Penalized at Step 6 ($v_6 = 0.3355$).	9	0.3355	20.0 (Incorrect)
118-c2 (Verifier Choice)	<i>Flawless derivation</i> : Initial vehicles $2 \times 5 = 10$; new vehicles $30 - 10 = 20$; containers $20/5 = 4$. Lowest step score remains high ($v_7 = 0.4843$).	8	0.4843	4.0 (Correct)
Case 2: False Attribution and Length Drift (<code>gsm8k-test-1043</code> , Gold Answer: 130000)				
1043-c0 (Majority Choice)	<i>Exhaustive derivation</i> : $26 \times 5 \times 1000 = 130000$. Step 12 summary received anomalous near-zero penalty ($v_{12} = 0.0159$).	12	0.0159	130000.0 (Correct)
1043-c2 (Verifier Choice)	<i>Severe arithmetic omission</i> : Omitted 5 cities per country ($26 \times 1000 = 26000$). Compact 7-step structure masked omission ($v_6 = 0.3576$).	7	0.3576	26000.0 (Incorrect)

Table 5: Exhaustive trajectory audit of representative test instances. Case 1 highlights accurate error localization over consensus. Case 2 exposes length-induced tail score degradation in the lightweight value head.

- **Verifier Intervention**: The verifier assigned step scores of $[0.994, 0.949, 0.843, 0.997, 0.981]$ to the initial steps, but dropped to $v_6 = \mathbf{0.3355}$ upon evaluating the invalid algebraic transition.
- **Winning Candidate (c2)**: Candidate c2 correctly computed the vehicle delta ($30 - 10 = 20$) and divided by 5 to obtain 4 containers. Its minimum step score was $v_7 = \mathbf{0.4843}$. Because $0.4843 > 0.3355$, Min aggregation successfully broke the invalid consensus, proving that step verification can rescue correct minority solutions.

Case 2: False Attribution via Length-Induced Drift (`gsm8k-test-1043`) The problem states: *"There are 26 countries in South America and, in each country, there are 5 cities with 1000 people living in each city... how many people are there in South America?"*

- **Gold Derivation (c0)**: Candidate c0 methodically multiplied $26 \times 5 = 130$ cities, and $130 \times 1000 = 130,000$ people. However, because each sentence was split into fine-grained sub-steps (12 steps total), the value head exhibited severe drift: while steps 1–9 scored above 0.72, the terminal sentence *"Thus, if the entire population consists of this total... the total population is 130,000"* was assigned an anomalous score of $v_{12} = \mathbf{0.0159}$.
- **Invalid Candidate (c2)**: Candidate c2 omitted the factor of 5 entirely, calculating $26 \times 1000 = 26,000$. However, because it completed its reasoning in only 7 concise steps, its lowest step score was $v_6 = \mathbf{0.3576}$.

- **Pathological Inversion**: Min aggregation compared the candidates via their minimum scores:

$$\mathcal{R}_{\min}(\text{c2}) = 0.3576 > 0.0159 = \mathcal{R}_{\min}(\text{c0})$$

The verifier selected the completely invalid candidate c2, discarding the mathematically flawless derivation c0.

6.2 Failure Taxonomy

Synthesizing observations across the 27 degradation instances, we categorize lightweight verification failures into four distinct classes:

- F1. Length-Induced Tail Degradation (44.4% of failures)**: As sequence length and step indices increase, the internal representations of sub-billion models suffer from positional degradation. The value head frequently misinterprets conversational wrap-ups or LaTeX formatting delimiters as logical breakdown, assigning severe penalties to terminal steps.
- F2. Superficial Fluency and Equation Hallucination (29.6% of failures)**: Compact value heads are susceptible to formula density bias. When an invalid candidate generates complex LaTeX environments (e.g., `\begin{align*}`), the verifier assigns high confidence scores based on structural surface features, failing to verify whether the arithmetic equality holds.
- F3. Step Segmentation Granularity Sensitivity (14.8% of failures)**: Step splitting based

on newline delimiters (`\n`) often isolates sentence fragments (e.g., bulleted lists) into standalone steps devoid of operational context. Without contextual self-containment, isolated fragments receive erratic value predictions.

F4. Parser Failure and Truncation Penalty (11.1% of failures): 15.2% of generated solutions could not be parsed by terminal regular expressions. When the verifier selected an unparseable candidate due to strong intermediate scores, it was automatically penalized as incorrect, whereas Majority Voting filtered unparseable solutions a priori.

7 Discussion and Critical Reflections

7.1 Why Did AutoPSV Work at Scale But Fail Here?

In original studies (Lu et al., 2024), AutoPSV yielded notable gains across Mistral-7B, Mixtral-8x7B, and Qwen-72B backbones. Our study establishes an empirical lower bound on model scale.

Representational Capacity Threshold Evaluating intermediate mathematical consistency is fundamentally more demanding than next-token language modeling. Detecting that $26 \times 1000 = 26000$ fails to satisfy the problem requirements demands tracking multi-entity constraints across multiple tokens. While a 70B parameter model maintains robust internal world states, a 0.5B model allocates its limited capacity to basic syntax and token prediction. Constraining optimization further via 4-bit quantization and low-rank LoRA subspace updates ($r = 8$) restricts the value head to surface-level lexical heuristics.

The Sample and Optimization Regimes The original AutoPSV framework trained outcome verifiers on upwards of 150,000 solution paths across GSM8K and MATH benchmarks (Lu et al., 2024). In our resource-constrained smoke test on HPC debugging nodes, we operated over 400 trajectories (5,302 prefix states) updated across 30 gradient update steps. While this proved sufficient to verify the mechanics of confidence extraction and adapter reloading, 30 gradient steps are insufficient for value head calibration, leaving predicted scores uncalibrated against ground-truth error rates.

7.2 Implications for Candidate Reranking Systems

Our negative results carry concrete engineering implications:

- **Voting Remains the Baseline to Beat:** In low-resource scenarios, consensus-based majority voting remains remarkably hard to surpass. Naive deployment of lightweight reward heads is actively detrimental, lowering baseline accuracy by over 8%.
- **Necessity of Length-Normalized Aggregation:** Step verifiers must incorporate length-normalization factors to prevent pathological score collapse on multi-step reasoning chains. Neither pure Min nor unnormalized Product is suitable for variable-length derivations.
- **Parser-Aware Verifier Decoding:** Verifiers must be constrained to score only candidates that satisfy basic terminal syntax and extraction guarantees.

8 Conclusion

In this work, we conducted a rigorous empirical replication and error analysis of automated process supervision (AutoPSV) within a lightweight computational regime. Using a 4-bit LoRA Qwen2.5-0.5B backbone on bwUniCluster HPC debugging nodes, we evaluated candidate reranking across 200 held-out GSM8K instances (1,000 candidates).

Our study provides an informative negative result: while the verifier demonstrates isolated capability in rescuing correct minority solutions from invalid consensus (11 problems), it induces substantial degradation overall (27 problems), trailing Majority Voting by 8.0% (Min) and 8.5% (Product). Through deep trajectory auditing, we identified length-induced score drift and formula surface bias as primary failure mechanisms. These findings delineate practical scaling boundaries, showing that automated process supervision requires sufficient backbone capacity and calibrated optimization to outperform simple frequency consensus.

Limitations

We explicitly delineate several structural limitations of this replication:

1. **Scale and Scope Bounds:** This investigation evaluated a lightweight adaptation (200 test

problems, 1,000 candidates, 0.5B backbone) and does not constitute a full-scale invalidation of the original AutoPSV paper (Lu et al., 2024). Full-parameter training on 7B+ backbones across 150,000 samples may overcome the representational thresholds identified here.

2. **Confounded Optimization Variables:** Due to HPC debugging queue limits (walltimes ≤ 1 hour), multiple efficiency techniques were combined simultaneously (4-bit NF4 quantization, LoRA with $r = 8$, 400 trajectories, 30 update steps). Decoupling the exact individual performance cost of quantization versus low trajectory volume remains an open area for future ablation.
3. **Single Benchmark Domain:** Empirical evaluation was confined to grade-school arithmetic (GSM8K). The behavior of lightweight process verifiers on commonsense reasoning or formal symbolic logic remains to be verified.

Ethical Considerations

This study utilizes public, open-source mathematical benchmark datasets (GSM8K) and openweights foundation models (Qwen2.5). The research poses no risks regarding personally identifiable information, toxic text generation, or dual-use societal harms. All compute was hosted on certified academic infrastructure.

References

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:1877–1901.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Pal: Program-aided language models. In *International Conference on Machine Learning (ICML)*, pages 10764–10799. PMLR.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*.
- Jianqiao Lu, Zhiyang Dou, Hongru Wang, Zeyu Cao, Jianbo Dai, Yingjia Wan, and Zhijiang Guo. 2024. AutoPSV: Automated process-supervised verifier. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.
- Peiyi Wang, Lei Li, Zhihong Shao, RX Xu, Damai Dai, Yifei Li, Deli Chen, Y Wu, and Zhifang Sui. 2023a. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. *arXiv preprint arXiv:2312.08935*.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023b. Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations (ICLR)*.
- Zihan Wang, Yunxuan Li, Yuexin Wu, Liangchen Luo, Le Hou, Hongkun Yu, and Jingbo Shang. 2024. Multi-step problem solving through a verifier: An empirical analysis on model-induced process supervision. *arXiv preprint arXiv:2402.02658*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems (NeurIPS)*.
- Fei Yu, Anningzhe Gao, and Benyou Wang. 2023. Outcome-supervised verifiers for planning in mathematical reasoning. *arXiv preprint arXiv:2311.09724*.

A Appendix: Detailed Data Structures and Implementation

A.1 Data Generation and Verification Logs

Figure 1 illustrates the end-to-end operational pipeline executed in this study, spanning trajectory generation, confidence extraction, value head fine-tuning, and test-time candidate reranking.

A.2 Step-Level Scoring Breakdown of Case 1

Table 6 provides the complete step-by-step scoring output generated by the Qwen2.5-0.5B LoRA value head on `gsm8k-test-118-c0` and `gsm8k-test-118-c2`.

A.3 Step-Level Scoring Breakdown of Case 2

Table 7 documents the step scores for `gsm8k-test-1043-c0` and `gsm8k-test-1043-c2`, demonstrating length-induced score collapse on long trajectories.

Pipeline Architecture Overview:

1. **Data Slicing & Trajectory Ingestion:** 400 GSM8K training trajectories segmented into 5,302 step-prefix representations.
2. **AutoPSV Automated Labeling:** Outcome-supervised confidence variations evaluated: $\Delta_{\text{conf}}^t = \frac{f_{\theta}(S^{(1:t+1)}) - f_{\theta}(S^{(1:t)})}{f_{\theta}(S^{(1:t)})}$. Pseudo-label threshold frozen at $\theta = -0.50$ via the first-error hypothesis.
3. **Quantized PEFT Fine-Tuning:** Base Qwen2.5-0.5B model frozen in 4-bit NF4 precision. Trainable LoRA matrices ($r = 8, \alpha = 16$) updated over 30 gradient steps on an NVIDIA A100 GPU (30GB memory). Checkpoint persistence confirmed with 0.0 reload error.
4. **Held-Out Generation:** Frozen generator produces 5 candidate chains across 200 out-of-sample problems ($T = 0.7, p = 0.9, 1,000$ total candidates).
5. **Reranking & Evaluation:** Verification via Min ($\min v_t$) and Product ($\sum \log v_t$) aggregation, benchmarked against Majority Voting and Oracle Pass@5.

Figure 1: System architecture and execution pipeline of the lightweight process-supervised verification benchmark.

Step	Candidate 118-c0 (Majority Choice: Incorrect)			Candidate 118-c2 (Verifier Choice: Correct)		
	Text Preview	v_t	Δ_{conf}^t	Text Preview	v_t	Δ_{conf}^t
1	Initially 2 containers of vehicles, each 5 vehicles	0.9944	–	Initially 2 containers, each 5 vehicles	0.9924	–
2	Initially, $2 \times 5 = 10$ vehicles were imported	0.9489	-0.046	2 cont. $\times$ 5 veh. = 10 veh.	0.9408	-0.052
3	Day 2 total number of vehicles was 30	0.8429	-0.112	Total number of vehicles is now 30	0.8670	-0.078
4	Let x be number of additional containers	0.9974	+0.183	Find additional vehicles: $30 - 10$	0.9305	+0.073
5	Total vehicles after importing x containers is 30	0.9809	-0.017	30 veh. $-$ 10 veh. = 20 veh.	0.8529	-0.083
6	Equation becomes $10 + x = 30$	0.3355	-0.658	Additional cont: divide 20 by 5	0.9344	+0.096
7	Subtract 10 from both sides: $x = 30 - 10$	0.8289	+1.471	$20/5 = 4$ containers	0.4843	-0.482
8	$x = 20$	0.6715	-0.190	Final boxed{4}	0.7402	+0.528
9	Containers imported on day 2 is boxed{20}	0.5361	-0.202	–	–	–
Min		0.3355			0.4843	

Table 6: Full step-level value head scores and confidence changes (Δ_{conf}^t) for gsm8k-test-118. In Step 6 of candidate c0, the model detects algebraic confusion, penalizing the step to 0.3355 ($\Delta_{\text{conf}}^t = -65.8\% < -50\%$).

Step	Candidate 1043-c0 (Majority Choice: Correct)			Candidate 1043-c2 (Verifier Choice: Incorrect)		
	Text Preview	v_t	Δ_{conf}^t	Text Preview	v_t	Δ_{conf}^t
1	Calculate number of cities: 26 countries	0.9161	–	Calculate total cities: 26 countries	0.6361	–
2	Each country has 5 cities	0.7387	-0.194	Total cities = 26	0.8278	+0.301
3	Population of one city: 1000 people	0.9890	+0.339	Population per city: 1000 people	0.9579	+0.157
4	In each city, 1000 people living	0.7210	-0.271	Population per city = 1000	0.6132	-0.360
5	Calculate total pop for cities in 1 country	0.8141	+0.129	Mult: cities $\times$ pop per city	0.6783	+0.106
6	Pop per city = 1000, cities = 5	0.7280	-0.106	$26 \times 1000 = 26000$	0.3576	-0.473
7	Pop per country = $1000 \times 5 = 5000$	0.9489	+0.303	Final boxed{26000}	0.9887	+1.765
8	Total pop in South America	0.9305	-0.019	–	–	–
9	$26 \times 5000 = 130000$ people	0.9073	-0.025	–	–	–
10	Summary context formulation	0.5250	-0.421	–	–	–
11	Summary wrap-up articulation	0.3549	-0.324	–	–	–
12	Final population is 130,000 people	0.0159	-0.955	–	–	–
Min		0.0159			0.3576	

Table 7: Full step-level value head scores and confidence changes (Δ_{conf}^t) for gsm8k-test-1043. Flawless candidate c0 suffers terminal score collapse to 0.0159 at step 12 ($\Delta_{\text{conf}}^t = -95.5\%$), causing pathological selection of invalid candidate c2.